

УЧЕБНИК ДЛЯ ТЕХ, КТО
НИКОГДА НЕ ПРОГРАММИРОВАЛ

Сайт с нуля

Как собрать интернет-магазин
и стать fullstack-разработчиком

60 глав · практика на живом коде

autodoc.tj

Содержание

1 Что такое сайт и что мы построим

Книга пишется по частям. Здесь — главы, готовые на сегодня.

ЧАСТЬ I. ЗНАКОМСТВО

Что такое сайт и что мы построим

ЗАЧЕМ ЭТО

Зачем эта глава

Люди бросают программирование не потому, что оно сложное. А потому, что первые две недели учат непонятно что и непонятно зачем. Учат «переменные», не понимая, где они в итоге окажутся.

Поэтому первая глава — не про код. Она про **карту местности**. Через полчаса вы будете понимать, из каких частей состоит любой сайт, кто их делает и чем эти люди отличаются. Дальше каждая новая тема будет ложиться на понятное место.

РАЗБИРАЕМСЯ

Сайт — это магазин

Забудьте на минуту про компьютеры. Представьте обычный магазин автозапчастей на улице.

В нём есть **три зоны**, и они очень разные:

1. **Торговый зал.** Полки, ценники, витрина, касса. Это то, что видит покупатель. Красиво разложено, подписано, всё понятно. Покупатель никогда не заходит дальше.
2. **Подсобка.** Туда покупателя не пускают. Там принимают товар, считают деньги, оформляют накладные, решают, кому какая скидка. Там работают сотрудники.
3. **Складской журнал.** Толстая тетрадь (или программа), где записано: какой товар, сколько штук, по какой цене, кто привёз, кому продали. Без неё магазин не магазин, а куча коробок.

Сайт устроен точно так же. Слово в слово:

В МАГАЗИНЕ	НА САЙТЕ	КАК ЭТО НАЗЫВАЮТ ПРОГРАММИСТЫ
Торговый зал	То, что видно в браузере	Фронтенд (frontend, «передняя часть»)
Подсобка	Программа на сервере	Бэкенд (backend, «задняя часть»)
Складской журнал	База данных	БД (database)

Человек, который умеет работать во всех трёх зонах, называется **fullstack-разработчик** («полный набор»). Это и есть цель книги.

РАЗБИРАЕМСЯ

Пять технологий и роль каждой

Чтобы построить магазин, нужно пять инструментов. Не пятьдесят — пять. Вот они, и вот зачем каждый:

► HTML — скелет

Определяет, что есть на странице: тут заголовок, тут картинка, тут кнопка, тут таблица. Никакой красоты — только содержание и структура.

Аналогия: **скелет человека**. Есть череп, рёбра, руки. Не очень симпатично, но понятно, где что.

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

Это уже рабочая страница. Уродливая, но рабочая.

► CSS — внешность

Отвечает за то, как это выглядит: цвета, шрифты, размеры, отступы, тени, расположение.

Аналогия: **одежда и внешность**. Тот же скелет, но теперь на нём мышцы, кожа и костюм.

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

Кнопка стала красной, белые буквы, скруглённые углы. Скелет тот же — вид другой.

► JavaScript — рефлекс в браузере

Реагирует на действия **прямо в браузере**, без перезагрузки страницы. Нажали «Купить» — счётчик корзины прибавился. Набрали в поиске три буквы — снизу выпали подсказки.

Аналогия: **рефлекс**. Дотронулись до горячего — рука отдёрнулась мгновенно, мозг ещё не успел подумать.

```
button.onclick = () ⇒ alert('Товар добавлен в корзину');
```

► PHP — мозг на сервере

Живёт **не в браузере, а на сервере** — на чужом компьютере, который работает круглосуточно. Решает всё важное: кто вошёл, что показать, сколько стоит, можно ли этому человеку в админку, записывать ли заказ.

Аналогия: **мозг и директор магазина в одном лице**. Принимает решения, к которым покупателя не подпускают.

```
if ($user['role'] === 'admin') {  
    echo 'Добро пожаловать в админку';  
}
```

Ключевая мысль, запомните её сразу: JavaScript выполняется на компьютере покупателя, и покупатель может его переписать. PHP выполняется на вашем сервере, и покупатель до него не дотянется. Поэтому цены, доступы и деньги считает **только PHP**. Никогда — JavaScript.

► SQL — память

Язык общения с базой данных. Спрашивает и записывает.

Аналогия: **разговор с кладовщиком**. «Дай все тормозные колодки дешевле 300 сомони, которые есть в наличии» — и он приносит список.

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

Выше вы увидели пять кусочков кода. Сейчас разберём **каждый значок**. Не запоминайте — просто прочитайте, чтобы код перестал выглядеть как шифр.

► HTML: почему всё в «уголках»

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
< и >	«уголки», угловые скобки	Внутри уголков — команда браузеру , а не текст для чтения. Браузер их не показывает
<h1>	«эйч-один»	Заголовок первого уровня , самый главный на странице. От английского <i>heading</i> — заголовок. Есть <h1> ... <h6>, чем больше цифра, тем мельче
</h1>	«закрывающий эйч-один»	Косая черта / означает конец . «Заголовок закончился здесь»
<p>	«пи»	Абзац обычного текста. От <i>paragraph</i>
<button>	«баттон»	Кнопка , на которую можно нажать
Текст между тегами		То, что реально увидит человек на экране

Пара «открывающий тег + содержимое + закрывающий тег» называется **элемент**. Это как скобки в математике: что открыли — то надо закрыть.

А что такое **<div>**? Вы это слово будете встречать чаще всех остальных. **<div>** — это **пустая коробка**. Он ничего не значит и никак не выглядит. Его задача — сгруппировать другие элементы, чтобы потом можно было сказать: «вот эту коробку покрасить в серый и сдвинуть вправо».

```
<div>
  <h1>Тормозные колодки</h1>
  <p>Цена: 250 сомони</p>
  <button>Купить</button>
</div>
```

Теперь заголовок, цена и кнопка лежат в одной коробке — это карточка товара. Название `div` — от английского *division*, «раздел». Подробно — в главе 5.

► CSS: почему фигурные скобки

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>	селектор	Кого красим. Здесь: все кнопки на странице
<code>{ }</code>	фигурные скобки	«Начало и конец списка указаний». Всё, что внутри, относится к кнопке
<code>background</code>	свойство	Что меняем — цвет фона
<code>:</code>	двоеточие	Разделитель: слева что меняем, справа на что
<code>#d92b2b</code>	значение	Цвет в виде кода. <code>#</code> + шесть символов — красный оттенок
<code>;</code>	точка с запятой	Конец одного указания. Дальше идёт следующее
<code>color: white</code>		Цвет текста — белый (не путать с <code>background</code> , это фон)
<code>border-radius: 8px</code>		Скругление углов на 8 пикселей

Формула CSS всегда одна: кого `{` что-меняем `:` на-что `;` `}`

► JavaScript: точки и стрелки

```
button.onclick = () => alert('Товар добавлен в корзину');
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>		Кнопка, которую мы раньше нашли на странице
<code>.</code>	точка	«Возьми у этого его свойство». Как «Иван ^{***} . ^{***} телефон» — телефон Ивана
<code>onclick</code>	«он-клик»	Свойство «что делать при нажатии». <i>on click</i> — «по щелчку»
<code>=</code>	равно, присваивание	Положи в левое то, что справа. Это не «равенство» из математики, а команда «запиши»
<code>() =></code>	«стрелочная функция»	«Вот действие , которое надо выполнить». Круглые скобки — место для входных данных, здесь их нет
<code>alert(...)</code>	«алерт»	Готовая команда браузера: показать всплывающее окошко
<code>'...'</code>	кавычки	Внутри кавычек — текст , а не команда. Компьютер не пытается его выполнять
<code>;</code>	точка с запятой	Конец строки-команды

► PHP: доллары и условия

```
if ($user['role'] === 'admin') {
    echo 'Добро пожаловать в админку';
}
```

Вот это и есть знаменитый `if` — самое важное слово в любом языке.

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
-----------	--------------	----------------

<code>if</code>	«иф», по-английски «если»	Проверка. «Если условие верно — сделай то, что в фигурных скобках. Если нет — пропусти»
<code>()</code>	круглые скобки после <code>if</code>	Внутри — условие, которое проверяем
<code>\$</code>	доллар	В PHP так помечают переменную — коробочку с данными. <code>\$user</code> — «коробочка с пользователем»
<code>\$user['role']</code>		Достать из коробочки <code>\$user</code> поле <code>role</code> (роль). Квадратные скобки — «возьми вот эту часть»
<code>===</code>	«строго равно»	Сравнение: «а точно ли слева то же самое, что справа?» Три знака, не один!
<code>'admin'</code>		Текст, с которым сравниваем
<code>{ }</code>	фигурные скобки	Тело условия — что делать, если проверка прошла
<code>echo</code>	«эхо»	Команда PHP: выведи это на страницу
<code>;</code>		Конец команды

Один `=` или три `===` ? Самая частая ошибка новичка:

- `=` — записать. `$x = 5` значит «положи пятёрку в коробочку x».
- `===` — спросить. `$x === 5` значит «а в коробочке правда пятёрка?»

Перепутаете — программа вместо проверки молча запишет значение.

А где же `else` ? `else` («иначе») — это вторая половина `if`. Читается как в жизни:

```
if ($user['role'] === 'admin') {  
    echo 'Добро пожаловать в админку';  
} else {  
    echo 'Вам сюда нельзя';  
}
```

Дословно: если роль администратор — покажи приветствие, иначе — откажи. Одно из двух выполнится обязательно. Подробно — в главе 21.

► SQL: запрос как предложение

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

SQL специально сделан похожим на английскую фразу. Читайте по кускам:

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
SELECT	«селект» — выбрать	Какие колонки нам нужны
name, price		Название и цена. Запятая — «и ещё»
FROM	«фром» — из	Из какой таблицы берём
products		Таблица товаров
WHERE	«вэа» — где, при условии	Какие строки брать, а какие пропустить
price < 300		Цена меньше 300
AND	«энд» — и	Оба условия должны выполняться разом
in_stock = 1		Есть в наличии (1 — да, 0 — нет)
;		Конец запроса

Целиком: «Выбери название и цену из таблицы товаров, где цена меньше 300 и товар в наличии». Почти обычное предложение.

► Что общего у всех пяти

Заметили? Значки повторяются из языка в язык:

ЗНАЧОК	ПОЧТИ ВЕЗДЕ ОЗНАЧАЕТ
{ }	«блок»: начало и конец группы
()	«сюда кладут данные»: условие или входные значения
;	«команда закончилась»
' ' или " "	«внутри — текст, не выполняй его»
// или #	комментарий: заметка для человека, компьютер её игнорирует

Выучив это один раз, вы будете узнавать их в любом языке — хоть в Python, хоть в Java.

РАЗБИРАЕМСЯ

Собираем картину целиком

Вот что происходит, когда покупатель открывает страницу товара. Прочитайте медленно — это самая важная схема в книге:

1. Покупатель нажимает ссылку «Тормозные колодки».
2. Браузер отправляет запрос на сервер: «дай мне страницу товара номер 42».
3. На сервере просыпается PHP и думает: «так, номер 42, надо узнать подробности».
4. PHP пишет запрос на SQL и отправляет в базу данных: `SELECT * FROM products WHERE id = 42`.
5. База отвечает: «Колодки тормозные, Bosch, 250 сомони, на складе 7 штук».
6. PHP берёт эти данные и **строит из них HTML** — вставляет название в заголовок, цену в ценник, картинку в блок картинки.
7. Готовый HTML улетает обратно в браузер.
8. Браузер читает HTML, применяет к нему CSS — и страница становится красивой.
9. Подключается JavaScript и начинает следить за кнопкой «Купить».



Как связаны части сайта

Все пять технологий сработали за доли секунды. Каждая сделала ровно свою часть.

Разделение обязанностей — вот главная идея всей веб-разработки. HTML не красит. CSS не считает. JavaScript не лезет в базу. PHP не рисует. Каждый занят своим делом.

РАЗБИРАЕМСЯ

Что именно мы построим

Не учебный «примерчик на три кнопки». Настоящий интернет-магазин автозапчастей — такой, каким пользуются живые люди.

К концу книги у вас будет работать:

Для покупателя:

- каталог запчастей с поиском по названию и артикулу;
- фильтры: бренд, цена, наличие;
- карточка товара с фото, описанием и характеристиками;

- корзина: добавить, изменить количество, удалить;
- оформление заказа с адресом и телефоном;
- личный кабинет: история заказов и их статусы.

Для продавца (магазин будет не только ваш — это маркетплейс):

- регистрация продавца;
- выкладка своих товаров;
- свои заказы и их обработка;
- баланс: сколько заработано, сколько уже выплачено.

Для администратора:

- модерация: проверять товары продавцов до публикации;
- управление всеми заказами и статусами;
- расчёт комиссии магазина;
- реестр выплат продавцам.

Плюс взрослые вещи, которых не бывает в учебниках:

- подключение внешнего API поставщика запчастей;
- пересчёт цены из рублей в сомони по живому курсу плюс наценка и доставка;
- защита от взлома: SQL-инъекции, XSS, кража сессии;
- выкладка сайта на настоящий хостинг под настоящим доменом с HTTPS.

РАЗБИРАЕМСЯ

Почему именно эти технологии

Честный ответ на вопрос «а не устарел ли PHP».

PHP выбран не потому, что он самый модный. А потому, что для первого сайта он **самый подходящий**, и вот почему:

- **Виден результат сразу.** Создал файл, открыл в браузере — работает. Не нужно собирать проект, ставить пятьдесят пакетов и настраивать сборщик.
- **Дешёвый хостинг.** PHP крутится на хостинге за 3-5 долларов в месяц. Это важно, когда вы делаете первый настоящий проект.
- **Огромная база примеров.** Любая ваша проблема уже решена и описана.
- **На нём работает больше половины сайтов в мире.** WordPress, а значит четверть всего интернета, — это PHP.

А главное: **навык не в языке**. Научившись думать «запрос → логика → база → ответ», вы пересядете на Python или Node.js за пару недель. Языки меняются, мышление остаётся.

ГРАБЛИ

Грабли новичка

«Сначала выучу всё, потом начну делать». Не работает. Знания без практики выветриваются за неделю. Правильно наоборот: делаем маленькую вещь → упираемся в незнание → узнаём ровно столько, сколько нужно → делаем дальше.

«Надо сразу выбрать самый лучший язык». Лучшего языка нет. Есть подходящий для задачи. Спор «PHP против Python» для новичка — это спор о том, каким молотком забивать первый гвоздь.

«Программисты знают всё наизусть». Нет. Программист с пятнадцатью годами опыта гуглит синтаксис по десять раз в день. Ценность не в памяти, а в умении разбить задачу на куски и понять, что искать.

«Я не математик, значит не смогу». Для веб-разработки нужна арифметика уровня седьмого класса. Складывать цены и считать проценты. Всё.

ЗАДАЧИ

Задачи

Задача 1.1. Откройте любой знакомый сайт (маркетплейс, новости, банк). Выпишите пять вещей, которые вы видите, — это работа фронтенда. Потом пять вещей, которые происходят невидимо (проверка пароля, расчёт доставки, сохранение заказа) — это бэкенд.

Задача 1.2. Для каждой строки определите, кто отвечает — HTML, CSS, JavaScript, PHP или SQL:

1. Кнопка стала синей вместо серой.
2. На странице появился второй заголовок.
3. При наборе букв в поиске снизу выпадают подсказки.
4. Проверка, что пароль при входе правильный.
5. Достать из базы 20 самых дешёвых товаров.
6. Меню на телефоне схлопывается в три полоски.
7. Заказ записался и ему присвоился номер.

Задача 1.3. Объясните вслух (правда вслух, это работает) своими словами разницу между фронтендом и бэкендом. Уложите в два предложения без слов «фронтенд» и «бэкенд».

Задача 1.4. Почему скидку постоянного покупателя нельзя считать на JavaScript? Ответ на одну строчку.

Задача 1.5. Нажмите на любом сайте правую кнопку мыши → «Посмотреть код страницы». То, что открылось, — это HTML, который прислал сервер. Найдите в нём заголовок страницы. Пока непонятно — это нормально, к главе 8 вы будете читать это свободно.

Ответы — в *Приложении Г*.

В БОЮ

В бою

Загляните в корень этого репозитория — там лежит реальный autodoc.tj. Пока просто посмотрите на названия папок, вы уже можете угадать, что где:

ПАПКА	ЧТО ВНУТРИ	КАКАЯ ЭТО ЗОНА
assets/css/	Файлы оформления	Фронтенд (CSS)
pages/	Страницы сайта	Фронтенд + бэкенд
includes/	Общая логика	Бэкенд (PHP)
sql/	Структура базы	База данных
admin/	Админка	Бэкенд, «подсобка»
seller/	Кабинет продавца	Бэкенд, «подсобка»
api/	Точки для JavaScript	Стык фронта и бэка

Тот самый магазин из аналогии. Торговый зал, подсобка и складской журнал.

ИТОГ

Итог главы

- Сайт = торговый зал (фронтенд) + подсобка (бэкенд) + журнал (база данных).

- **HTML** — скелет, **CSS** — внешность, **JavaScript** — рефлексy в браузере, **PHP** — мозг на сервере, **SQL** — разговор с памятью.
- JavaScript выполняется у покупателя, PHP — у вас. Деньги и доступы считает только PHP.
- Fullstack — тот, кто работает во всех трёх зонах.
- Учиться нужно от задачи к теории, а не наоборот.



Продолжение следует.